

## Chapitre 6 : Interrogation de la base de données (SELECT)

### I. Introduction

Ce chapitre traite de l'aspect le plus connu du langage SQL qui concerne l'extraction des données par requêtes (nom donné aux instructions SELECT). Une requête permet de rechercher des données dans une ou plusieurs tables ou vues, à partir de critères simples ou complexes.

### II. SELECT

L'instruction SELECT est une commande déclarative (elle décrit ce que l'on cherche sans expliquer le moyen d'opérer).

La restriction qui est programmée dans le WHERE de la requête permet de restreindre la recherche à une ou plusieurs lignes. Dans notre exemple une restriction répond à la question : « Quels sont les avions de type A320 ? ».

- **La projection** qui est programmée dans le SELECT de la requête permet d'extraire une ou plusieurs colonnes. Dans notre exemple elle répond à la question : « Quels sont les numéros de brevet et les nombres d'heures de vol de tous les pilotes ? ».
- **La jointure** qui est programmée dans le WHERE de la requête permet d'extraire des données de différentes tables en les reliant deux à deux (le plus souvent à partir de contraintes référentielles). Dans notre exemple la première jointure répond à la question « Quels sont les numéros de brevet et nombres d'heures de vol des pilotes de la compagnie de nom AirFrance ? ». La deuxième jointure répond à la question : « Quels sont les avions de la compagnie de nom Air France ? ».

En combinant ces trois fonctionnalités, toute question logique devrait trouver en théorie une réponse par une ou plusieurs requêtes.

#### Syntaxe (SELECT)

Pour pouvoir extraire des enregistrements d'une table, il faut que celle-ci soit dans votre base ou que vous ayez reçu le privilège SELECT sur la table.

La syntaxe SQL simplifiée de l'instruction SELECT est la suivante :

```
SELECT [ { DISTINCT | DISTINCTROW } | ALL ] listeColonnes  
FROM nomTable1 [,nomTable2]... [ WHERE condition ][clauseRegroupement ]  
[ HAVING condition ][ clauseOrdonnancement ]  
[ LIMIT [rangDépart,] nbLignes ] ;
```

Nous détaillerons chaque option à l'aide d'exemples au cours de ce chapitre.

Étudions la partie de l'instruction SELECT qui permet de programmer l'opérateur de projection (surligné dans la syntaxe suivante) :

### SELECT

FROM *nomTable* [*aliasTable*]

[ *clauseOrdonnancement* ]

[ LIMIT [*rangDépart*,] *nbLignes* ];

\_ DISTINCT et DISTINCTROW jouent le même rôle, à savoir ne pas inclure les duplicatas.

\_ ALL prend en compte les duplicatas (option par défaut).

\_ *listeColonnes* : { \* | *expression1* [[AS] *alias1* ] [, *expression2* [[AS] *alias2* ]...}

\_ \* : extrait toutes les colonnes de la table.

\_ *expression* : nom de colonne, fonction SQL, constante ou calcul.

\_ *alias* : renomme l'expression (nom valable pendant la durée de la requête).

\_ FROM désigne la table (qui porte un alias ou non) à interroger.

\_ *clauseOrdonnancement* : tri sur une ou plusieurs colonnes ou expressions.

\_ LIMIT pour limiter le nombre de lignes après résultat.

### 1. Extraction de toutes les colonnes

L'extraction de toutes les colonnes d'une table requiert l'utilisation du symbole « \* ».

Tableau 1 Utilisation de « \* »

| Requête SQL     | Résultat |                  |         |         |
|-----------------|----------|------------------|---------|---------|
|                 | +-----+  | +-----+          | +-----+ | +-----+ |
|                 | brevet   | nom              | nbHVol  | compa   |
|                 | +-----+  | +-----+          | +-----+ | +-----+ |
| mysql> SELECT * | PL-1     | Gratien Viel     | 450.00  | AF      |
| FROM Pilote;    | PL-2     | Didier Donsez    | 0.00    | AF      |
|                 | PL-3     | Richard Grin     | 1000.00 | SING    |
|                 | PL-4     | Placide Fresnais | 2450.00 | CAST    |
|                 | PL-5     | Daniel Vielle    | NULL    | AF      |
|                 | +-----+  | +-----+          | +-----+ | +-----+ |

### 2. Extraction de certaines colonnes

La liste des colonnes à extraire se trouve dans la clause SELECT.

Tableau 2 Liste de colonnes

| Requête SQL                                    | Résultat                                                                                                                                                                 |
|------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>SELECT compa, brevet FROM Pilote;</code> | <pre> +-----+-----+   compa   brevet   +-----+-----+   AF      PL-1       AF      PL-2       SING    PL-3       CAST    PL-4       AF      PL-5     +-----+-----+ </pre> |

### 2.1.Alias

Les alias permettent de renommer des colonnes à l’affichage ou des tables dans la requête. Les alias de colonnes sont utiles pour les calculs.

Tableau 3 Alias (colonnes et tables)

| Alias de colonnes                                                                                                                                                                                                                                                                        | Alias de table                                                                                                                                                                                                     |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>SELECT compa AS c1,<br/>nom AS NometPrenom, brevet c3<br/>FROM Pilote;</code>                                                                                                                                                                                                      | <code>SELECT aliasPilotes.compa AS<br/>aliasPilotes.nom<br/>FROM Pilote aliasPilotes;</code>                                                                                                                       |
| <pre> +-----+-----+-----+   c1      NometPrenom   c3      +-----+-----+-----+   AF      Gratien Viel   PL-1       AF      Didier Donsez   PL-2       SING    Richard Grin   PL-3       CAST    Placide Fresnais   PL-4       AF      Daniel Vielle   PL-5     +-----+-----+-----+ </pre> | <pre> +-----+-----+   c1      nom            +-----+-----+   AF      Gratien Viel     AF      Didier Donsez     SING    Richard Grin     CAST    Placide Fresnais     AF      Daniel Vielle   +-----+-----+ </pre> |

### 2.2.Duplicatas

Les directives **DISTINCT** ou **DISTINCTROW** éliminent les éventuels duplicatas. Pour la deuxième requête, les écritures « **DISTINCT compa** », « **DISTINCTROW (compa)** » et « **DISTINCTROW compa** » sont équivalentes. La notation entre parenthèses est nécessaire lorsque l’on désire éliminer des duplicatas par paires, triplets, etc.

Tableau 4 Gestion des duplicatas

| Avec duplicata                                                                                   | Sans duplicata                                                               |
|--------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------|
| <code>SELECT compa FROM Pilote;</code>                                                           | <code>SELECT <b>DISTINCT</b>(compa) FROM Pilote;</code>                      |
| <pre> +-----+   compa   +-----+   AF        AF        SING      CAST      AF      +-----+ </pre> | <pre> +-----+   compa   +-----+   AF        SING      CAST    +-----+ </pre> |

### 2.3. Ordonnement

Pour trier le résultat d'une requête, il faut spécifier la clause d'ordonnement par ORDER BY de la manière suivante :

#### ORDER BY

{ *expression1* | *position1* | *alias1* } [ASC | DESC]

[, { *expr2* | *position2* | *alias2* } [ASC | DESC]

\_ *expression* : nom de colonne, fonction SQL, constante, calcul.

\_ *position* : entier qui désigne l'expression (au lieu de la nommer) dans son ordre d'apparition dans la clause SELECT.

\_ ASC ou DESC : tri ascendant ou descendant (par défaut ASC).

Dans l'exemple suivant, on remarque que la valeur NULL est considérée comme plus petite que 0.

Tableau 5 Ordonnement

| Options par défaut                                                                                                                                                                                                     | Ordre décroissant (et NULL)                                                                                                                                                                      |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| mysql> SELECT brevet, nom<br>FROM Pilote ORDER BY nom;                                                                                                                                                                 | mysql> SELECT brevet, nbHVol<br>FROM Pilote ORDER BY nbHVol DESC;                                                                                                                                |
| <pre> +-----+-----+   brevet   nom        +-----+-----+   PL-5     Daniel Vielle     PL-2     Didier Donsez     PL-1     Gratien Viel      PL-4     Placide Fresnais     PL-3     Richard Grin    +-----+-----+ </pre> | <pre> +-----+-----+   brevet   nbHVol      +-----+-----+   PL-4     2450.00       PL-3     1000.00       PL-1     450.00        PL-2     0.00          PL-5     NULL        +-----+-----+ </pre> |

### 2.4. Concaténation

L'opérateur de concaténation se programme à l'aide de la fonction CONCAT qui admet deux chaînes de caractères en paramètre. Cette fonction permet de concaténer différentes expressions (colonnes, calculs, résultats de fonctions SQL ou constantes), sous réserve d'éventuelles conversions (*casting*). La colonne résultante est considérée comme une chaîne de caractères.

L'exemple suivant présente un alias dans l'en-tête de colonne ("Embauche") qui met en forme les résultats. La concaténation concerne deux colonnes et la constante "vole pour".

Tableau 6 Concaténation

| Requête                                                                           | Résultat                                                                                                                                                                                                                                                                                  |
|-----------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| SELECT brevet,<br>CONCAT(nom, ' vole pour ', compa)<br>AS "Embauche" FROM Pilote; | <pre> +-----+-----+   brevet   Embauche   +-----+-----+   PL-1     Gratien Viel vole pour AF     PL-2     Didier Donsez vole pour AF     PL-3     Richard Grin vole pour SING     PL-4     Placide Fresnais vole pour CAST     PL-5     Daniel Vielle vole pour AF   +-----+-----+ </pre> |

### 2.5.Insertion multiligne

Nous pouvons maintenant décrire l'insertion multiligne évoquée au chapitre précédent. Dans l'exemple suivant, il s'agit d'insérer tous les pilotes de la table Pilote (en considérant le nom, le nombre d'heures de vol et la compagnie) dans la table NomsetHVoldesPilotes :

Tableau 7 Insertion multiligne

| Création et insertion                                                                                | Requête et résultat                                                                                                                                                                                                                                                                          |
|------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CREATE TABLE<br>NomsetHVoldesPilotes<br>(nom VARCHAR(16),<br>nbHVol DECIMAL(7,2),<br>compa CHAR(4)); | mysql> SELECT * FROM NomsetHVoldesPilotes;                                                                                                                                                                                                                                                   |
| INSERT INTO NomsetHVoldesPilotes<br>SELECT nom,nbHVol,compa<br>FROM Pilote;                          | <pre> +-----+-----+-----+   nom            nbHVol   compa   +-----+-----+-----+   Gratien Viel   450.00   AF        Didier Donsez   0.00     AF        Richard Grin   1000.00   SING      Placide Fresnais   2450.00   CAST      Daniel Vielle   NULL     AF      +-----+-----+-----+ </pre> |

Notez que les instructions (CREATE TABLE et INSERT...) peuvent être programmées en une instruction (option AS SELECT de la commande CREATE TABLE) :

```
CREATE TABLE NomsetHVoldesPilotes AS SELECT nom, nbHVol, compa
FROM Pilote;
```

Limitation du nombre de lignes

Pour limiter le nombre de lignes à extraire à partir du résultat d'une requête, il faut spécifier la clause LIMIT de la manière suivante :

**LIMIT** [rangDépart,] nbLignes

Le premier entier précise le rang de la première ligne sélectionnée (en fonction du tri du résultat).

Le second entier indique le nombre maximum de lignes à extraire. La première ligne est considérée comme présente au rang 0. Ainsi « LIMIT n » équivaut à « LIMIT 0,n ».

L'exemple suivant illustre deux utilisations de la clause LIMIT :

Tableau 8 Limitation des résultats

| Requête                                                                       | Résultat                                                                                                                                                                                                                            |
|-------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Deuxième et troisième (ordre de clé) pilote.                                  |                                                                                                                                                                                                                                     |
| SELECT * FROM Pilote <b>LIMIT 1,2;</b>                                        | <pre> +-----+-----+-----+-----+   brevet   nom             nbHVol   compa   +-----+-----+-----+-----+   PL-2     Didier Donsez   0.00     AF        PL-3     Richard Grin    1000.00   SING    +-----+-----+-----+-----+ </pre>     |
| Les deux pilotes les plus expérimentés (par ordre du nombre d'heures de vol). |                                                                                                                                                                                                                                     |
| SELECT * FROM Pilote ORDER BY nbHvol DESC <b>LIMIT 2;</b>                     | <pre> +-----+-----+-----+-----+   brevet   nom             nbHVol   compa   +-----+-----+-----+-----+   PL-4     Placide Fresnais   2450.00   CAST      PL-3     Richard Grin    1000.00   SING    +-----+-----+-----+-----+ </pre> |

### III. Restriction (WHERE)

Les éléments de la clause WHERE d'une requête permettent de programmer l'opérateur de restriction. Cette clause limite la recherche aux enregistrements qui respectent une condition simple ou complexe. Cette section s'intéresse à la partie surlignée de l'instruction SELECT suivante :

```
SELECT [ { DISTINCT | DISTINCTROW } | ALL ] listeColonnes
FROM nomTable [aliasTable]
```

;

*\_condition* composée de colonnes, d'expressions, de constantes liées deux à deux entre des opérateurs :

- de comparaison (>, =, <, >=, <=, <>);
- logiques (NOT, AND ou OR);
- intégrés (BETWEEN, IN, LIKE, IS NULL).



Tableau 9 Opérateurs logiques

| Requête                                                                                                         | Résultat sous SQL*Plus                                                                                                                                                                                                                                 |
|-----------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>SELECT brevet, nom, compa FROM Pilote WHERE ( compa = 'SING' OR compa = 'AF' AND nbHVol &lt; 500);</pre>   | <pre>+-----+-----+-----+   brevet   nom            compa   +-----+-----+-----+   PL-1     Gratien Viel   AF         PL-2     Didier Donsez   AF         PL-3     Richard Grin   SING       PL-5     Daniel Vielle   SING     +-----+-----+-----+</pre> |
| <pre>SELECT brevet, nom, compa FROM Pilote WHERE ( (compa = 'SING' OR compa = 'AF') AND nbHVol &lt; 500);</pre> | <pre>+-----+-----+-----+   brevet   nom            compa   +-----+-----+-----+   PL-1     Gratien Viel   AF         PL-2     Didier Donsez   AF         PL-5     Daniel Vielle   SING     +-----+-----+-----+</pre>                                    |

### 1. Opérateurs intégrés

Les opérateurs intégrés sont BETWEEN, IN, LIKE et IS NULL.

Tableau 10 Opérateurs intégrés

| Opérateur                                                                                             | Exemple                                                                                                                                                                                                                                                                                                                                       |
|-------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>BETWEEN <i>limiteInf</i> AND <i>limiteSup</i> teste l'appartenance à un intervalle de valeurs.</p> | <pre>SELECT brevet, nom, nbHVol FROM Pilote WHERE nbHVol BETWEEN 399 AND 1000;</pre> <pre>+-----+-----+-----+   brevet   nom            nbHVol   +-----+-----+-----+   PL-1     Gratien Viel   450.00     PL-3     Richard Grin   1000.00     PL-5     Daniel Vielle   400.00   +-----+-----+-----+</pre>                                     |
| <p>IN (<i>listeValeurs</i>) compare une expression avec une liste de valeurs.</p>                     | <pre>SELECT brevet, nom, compa FROM Pilote WHERE compa IN ('CAST', 'SING');</pre> <pre>+-----+-----+-----+   brevet   nom            compa   +-----+-----+-----+   PL-3     Richard Grin   SING       PL-4     Placide Fresnais   CAST       PL-5     Daniel Vielle   SING       PL-6     Francoise Tort   CAST     +-----+-----+-----+</pre> |

LIKE (*expression*) compare de manière générique des chaînes de caractères à une *expression*.

Le symbole % remplace un ou plusieurs caractères.

Le symbole \_ remplace un caractère.

Ces symboles peuvent se combiner.

Utilisez de préférence des colonnes

VARCHAR ou complétez si nécessaire par des blancs jusqu'à la taille maximale pour des CHAR.

```
SELECT brevet, nom, compa FROM Pilote
WHERE compa LIKE ('%A%');
```

| brevet | nom              | compa |
|--------|------------------|-------|
| PL-1   | Gratien Viel     | AF    |
| PL-2   | Didier Donsez    | AF    |
| PL-4   | Placide Fresnais | CAST  |
| PL-6   | Francoise Tort   | CAST  |

```
SELECT brevet, nom, compa FROM Pilote
WHERE compa LIKE ('A_');
```

| brevet | nom           | compa |
|--------|---------------|-------|
| PL-1   | Gratien Viel  | AF    |
| PL-2   | Didier Donsez | AF    |

IS NULL compare une *expression* (colonne, calcul, constante) à la valeur NULL.

La négation s'écrit soit « *expression* IS NOT NULL » soit « NOT (*expression* IS NULL) ».

```
SELECT nom, prime, nbHVol FROM Pilote
WHERE prime IS NULL OR nbHVol IS NULL;
```

| nom            | prime | nbHVol |
|----------------|-------|--------|
| Didier Donsez  | NULL  | 0.00   |
| Francoise Tort | 0     | NULL   |

## 2. Fonctions

MySQL propose un grand nombre de fonctions qui s'appliquent dans les clauses SELECT ou WHERE d'une requête. La syntaxe générale d'une fonction est la suivante :

*nomFonction*(*colonne1* | *expression1* [, *colonne2* | *expression2* ...])

- Une fonction monoligne agit sur une ligne à la fois et ramène un résultat par ligne. On distingue quatre familles de fonctions monolignes : caractères, numériques, dates et conversions de types de données. Ces fonctions peuvent se combiner entre elles (exemple : MAX(COS(ABS(n))) désigne le maximum des cosinus de la valeur absolue de la colonne n).
- Une fonction multiligne (fonction d'agrégat) agit sur un ensemble de lignes pour ramener un résultat (voir la section *Regroupements*).

La plupart des fonctions pour les caractères acceptent une chaîne de caractères en paramètre de nature CHAR ou VARCHAR. Le tableau suivant décrit les principales fonctions :



Tableau 11 Fonctions pour les caractères

| Fonction                                                | Objectif                                                                                                                                | Exemple                                                                                                                                                                         |
|---------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ASCII( <i>c</i> )                                       | Retourne le caractère ASCII équivalent.                                                                                                 | <b>ASCII</b> ('A') donne 65                                                                                                                                                     |
| CHAR( <i>n</i> )                                        | Retourne le caractère équivalent dans le jeu de caractères en cours.                                                                    | <b>CHR</b> (161)    <b>CHR</b> (162) donne íó                                                                                                                                   |
| CONCAT( <i>c1</i> , <i>c2</i> )                         | Concatène deux chaînes.                                                                                                                 | <pre>SELECT <b>CONCAT</b>( <b>CONCAT</b>(nom, ' vole pour '), compa) "Personnel" FROM Pilote;</pre> <pre>+-----+   Personnel   +-----+   viel travaille pour AF     ...  </pre> |
| FIELD( <i>c</i> , <i>c1</i> , <i>c2</i> ...)            | Retourne l'index qui correspond à la première égalité entre <i>c</i> et <i>c1</i> , <i>c2</i> , etc. 0 si aucune égalité n'est trouvée. | <pre>SELECT <b>FIELD</b>('Air', 'air', 'Airbus', 'Air') "Attention à la casse!";</pre> <pre>+-----+   Attention à la casse!   +-----+   1  </pre>                               |
| INSERT( <i>c1</i> , <i>pos</i> , <i>t</i> , <i>c2</i> ) | Modifie la chaîne <i>c1</i> en insérant <i>t</i> caractères de la sous-chaîne <i>c2</i> à partir de la position <i>pos</i> .            | <pre>SELECT <b>INSERT</b>('Compxxxie : Airbus ', 5, 3, 'agn') "Qui?";</pre> <pre>+-----+   Qui?   +-----+   Compagnie : Airbus   +-----+</pre>                                  |
| INSTR( <i>c1</i> , <i>c2</i> )                          | Premier indice d'une sous-chaîne <i>c1</i> dans une chaîne <i>c2</i> .<br>Exemple : indice de 'Air' dans la chaîne.                     | <pre>SELECT <b>INSTR</b>('Infos-Air : AirBus pour Air-France', 'Air') "Indice";</pre> <pre>+-----+   Indice   +-----+   7  </pre>                                               |

|                      |                                                                                                                                                                    |                                                                                                                                                                                                            |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| LOWER (c)            | Tout en minuscules.                                                                                                                                                | <pre>SELECT CONCAT(LOWER(prenom), ' ',               LOWER(nom)) "Etat civil" FROM Pilote WHERE compa = 'SING';</pre> <pre>+-----+   Etat civil   +-----+   richard grin     daniel vielle   +-----+</pre> |
| LOCATE (c1, c2, pos) | Premier indice d'une sous-chaîne c1 dans une chaîne c2 à partir de la position pos. Exemple : indice de 'Air' dans la chaîne à partir du 9 <sup>e</sup> caractère. | <pre>SELECT LOCATE('Air', 'Infos-Air : Airbus pour Air-France', 9) "Indice après 9";</pre> <pre>+-----+   Indice après 9   +-----+   13   +-----+</pre>                                                    |
| LENGTH (c)           | Longueur de la chaîne.                                                                                                                                             | <pre>SELECT LENGTH('Infos-Air : Airbus pour Air-France') "Taille";</pre> <pre>+-----+   Taille   +-----+   35   +-----+</pre>                                                                              |
| LEFT (c, n)          | Extrait les n premiers caractères à c en partant de la gauche.                                                                                                     | <pre>SELECT LEFT('A380 à BlagnacB747B747', 14) "Bye les Jumbo";</pre> <pre>+-----+   Bye les Jumbo   +-----+   A380 à Blagnac   +-----+</pre>                                                              |
| LPAD (c1, n, c2)     | Insertion à gauche de c2 dans c1 sur n caractères.                                                                                                                 | <pre>SELECT LPAD('Rien', 20, '-.-') "sur 20";</pre> <pre>+-----+   sur 20   +-----+   -.-.-.-.-.-.-.-Rien   +-----+</pre>                                                                                  |
| REPLACE (c1, c2, c3) | Recherche les c2 présentes dans c1 et les remplace par c3.                                                                                                         | <pre>SELECT   REPLACE('Matra et Aerospatiale',   'Matra', 'EADS') "Changement";</pre> <pre>+-----+   Changement   +-----+   EADS et Aerospatiale   +-----+</pre>                                           |

|                   |                                                                                                                                                                                                                    |                                                                                                                                                                                                                                                                                |
|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| REVERSE (c)       | Retourne la chaîne renversée.                                                                                                                                                                                      | <pre>SELECT REVERSE('cangalB à 083A')       "Miroir";</pre> <pre>+-----+   Miroir   +-----+   A380 à Blagnac   +-----+</pre>                                                                                                                                                   |
| RIGHT (c,n)       | Extrait les n derniers caractères à c en partant de la droite.                                                                                                                                                     | <pre>SELECT RIGHT('B747B747A380 à Blagnac',14)       "Bye les Jumbo";</pre> <pre>+-----+   Bye les Jumbo   +-----+   A380 à Blagnac   +-----+</pre>                                                                                                                            |
| RPAD (c1,n,c2)    | Insertion à droite de c2 dans c1 sur n caractères.                                                                                                                                                                 | <pre>SELECT RPAD('Rien',19,'-.-') "sur 19";</pre> <pre>+-----+   sur 19   +-----+   Rien-.-.-.-.-.-.-.-.-.-.-   +-----+</pre>                                                                                                                                                  |
| SOUNDEX (c)       | Extrait la phonétique d'une expression ( <i>in english only</i> !).                                                                                                                                                | <pre>SELECT nom, surnom, compa FROM Pilote       WHERE SOUNDEX(surnom) IN       ( SOUNDEX('SMYTHE'), SOUNDEX('John') );</pre> <pre>+-----+-----+-----+   nom   surnom   compa   +-----+-----+-----+   donsez   smith   AF     vielle   jone   SING   +-----+-----+-----+</pre> |
| SUBSTR (c,n,[t])  | Extraction de la sous-chaîne c commençant à la position n sur t caractères.                                                                                                                                        | <pre>SELECT       SUBSTR('Air France à Blagnac Con!',12,9)       "Où ça?";</pre> <pre>+-----+   Où ça?   +-----+   à Blagnac   +-----+</pre>                                                                                                                                   |
| TRIM (c1 FROM c2) | Enlève les caractères c1 à la chaîne c2 (options LEADING et TRAILING pour préciser le sens du découpage). Existent aussi LTRIM et RTRIM qui enlèvent des espaces respectivement au début ou à la fin d'une chaîne. | <pre>SELECT       TRIM('B' FROM 'BA380 à BlagnacBBBBB')       "Bye les Jumbo";</pre> <pre>+-----+   Bye les Jumbo   +-----+   A380 à Blagnac   +-----+</pre>                                                                                                                   |

UPPER

Tout en majuscules.

```
SELECT CONCAT(UPPER(prenom), ' ',
UPPER(nom)) "Pilotes de CAST"
FROM Pilote WHERE compa = 'CAST';
```

| +-----+ |                  |
|---------|------------------|
|         | Pilotes de CAST  |
| +-----+ |                  |
|         | PLACIDE FRESNAIS |
|         | FRANCOISE TORT   |
| +-----+ |                  |

### 3. Numériques

En plus des opérateurs arithmétiques disponibles dans les langages de programmation (+, -, \*, / et DIV pour la division entière), MySQL supporte un grand nombre de fonctions numériques.

Tableau 12 Fonctions numériques

| Fonction                                 | Objectif                                                                    | Exemple                                            |
|------------------------------------------|-----------------------------------------------------------------------------|----------------------------------------------------|
| ABS ( <i>n</i> )                         | Valeur absolue de <i>n</i> .                                                |                                                    |
| ACOS ( <i>n</i> )                        | Arc cosinus ( <i>n</i> de -1 à 1), retour exprimé en radians (de 0 à pi).   |                                                    |
| ATAN ( <i>n</i> )                        | Arc tangente ( $\forall n$ ), retour exprimé en radians ( de -pi/2 à pi/2). |                                                    |
| CEIL ( <i>n</i> )                        | Plus petit entier $\geq$ à <i>n</i> .                                       | <b>CEIL</b> (15.7) retourne 16.                    |
| COS ( <i>n</i> )                         | Cosinus de <i>n</i> exprimé en radians de 0 à 2 pi.                         | <b>COS</b> (60*PI()/180) retourne 0.5.             |
| COT ( <i>n</i> )                         | Cotangente de <i>n</i> exprimée en radians.                                 | <b>COT</b> (30*PI()/180) retourne 1.7320508075689. |
| DEGREES ( <i>n</i> )                     | Conversion de radians en degrés.                                            | <b>DEGREES</b> (PI()/2) retourne 90.               |
| EXP ( <i>n</i> )                         | e (2.71828183) à la puissance <i>n</i> .                                    |                                                    |
| FLOOR ( <i>n</i> )                       | Plus grand entier $\leq$ à <i>n</i> .                                       | <b>FLOOR</b> (15.7) retourne 15.                   |
| LN ( <i>n</i> )                          | Logarithme népérien de <i>n</i> .                                           |                                                    |
| LOG ( <i>n</i> ) ( <i>m</i> , <i>n</i> ) | Logarithme de <i>n</i> dans une base <i>m</i> .                             |                                                    |
| MOD ( <i>m</i> , <i>n</i> )              | Reste de la division entière de <i>m</i> par <i>n</i> .                     |                                                    |
| POW ( <i>m</i> , <i>n</i> )              | <i>m</i> puissance <i>n</i> .                                               |                                                    |
| RADIANS ( <i>n</i> )                     | Conversion de degrés en radians.                                            | <b>RADIANS</b> (90) retourne 1.5707963267949.      |
| RAND ()                                  | Flottant aléatoire (à 14 décimales) entre 0 et 1.                           | <b>ROUND</b> (17.567,2) retourne 17,57.            |
| ROUND ( <i>m</i> , <i>n</i> )            | Arrondi à une ou plusieurs décimales.                                       | <b>ROUND</b> (17.567,2) retourne 17,57.            |
| SIGN ( <i>n</i> )                        | Retourne le signe d'un nombre (-1, 0 ou 1).                                 |                                                    |
| SIN ( <i>n</i> )                         | Sinus de <i>n</i> exprimé en radians de 0 à 2 pi.                           | <b>SIN</b> (30*PI()/180) retourne 0.5.             |
| SINH ( <i>n</i> )                        | Sinus hyperbolique de <i>n</i> .                                            |                                                    |
| SQRT ( <i>n</i> )                        | Racine carrée de <i>n</i> .                                                 |                                                    |
| TAN ( <i>n</i> )                         | Tangente de <i>n</i> exprimée en radians de 0 à 2 pi.                       |                                                    |
| TRUNCATE ( <i>n</i> , <i>m</i> )         | Coupe de <i>n</i> à <i>m</i> décimales.                                     | <b>TRUNC</b> (15.79,1) retourne 15.7.              |



#### 4. Dates

Le tableau suivant décrit les principales fonctions pour les dates :

Tableau 13 Fonctions pour les dates

| Fonction                                             | Objectif                                                                                                                                                              | Retour           |
|------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|
| ADDDATE ( <i>date</i> , <i>n</i> )                   | Ajoute <i>n</i> jours à une date (heure).                                                                                                                             | DATE ou DATETIME |
| ADDTIME ( <i>date1</i> , <i>date2</i> )              | Ajoute les deux dates avec <i>date1</i> TIME ou DATETIME, et <i>date2</i> TIME.                                                                                       | TIME ou DATETIME |
| CURDATE (), CURRENT_DATE ou CURRENT_DATE ()          | Date courante ('YYYY-MM-DD' ou YYYYMMDD).                                                                                                                             | INT ou DATE      |
| CURTIME (), CURRENT_TIME ou CURRENT_TIME ()          | Heure courante ('HH:MM:SS' or HHMMSS).                                                                                                                                | INT ou DATE      |
| CURRENT_TIMESTAMP, CURRENT_TIMESTAMP () ou NOW ()    | Date et heure courantes ('YYYY-MM-DD HH:MM:SS' ou YYYYMMDDHHMMSS).                                                                                                    | INT ou DATETIME  |
| DATE ( <i>date</i> )                                 | Extrait une date à partir d'une expression de type DATETIME.                                                                                                          | DATE             |
| DATEDIFF ( <i>date1</i> , <i>date2</i> )             | Nombre entier de jours entre les 2 dates.                                                                                                                             | INT              |
| DATE_ADD ( <i>date</i> , INTERVAL <i>expr type</i> ) | Ajoute un intervalle à une date (heure). <i>expr</i> désigne un intervalle. <i>type</i> indique comment interpréter le format de l'expression (voir tableau suivant). | DATE ou DATETIME |
| DATE_FORMAT ( <i>date</i> , <i>format</i> )          | Présente la date selon un format (voir tableau suivant).                                                                                                              | VARCHAR          |
| DATE_SUB ( <i>date</i> , INTERVAL <i>expr type</i> ) | Soustrait un intervalle à une date (heure) Mêmes paramètres que DATE_ADD.                                                                                             | DATE ou DATETIME |
| DAYNAME ( <i>date</i> )                              | Nom du jour en anglais.                                                                                                                                               | VARCHAR          |
| DAY( <i>date</i> ) ou DAYOFMONTH ( <i>date</i> )     | Numéro du jour dans le mois (0 à 31).                                                                                                                                 | INT              |
| DAYOFYEAR ( <i>date</i> )                            | Numéro du jour dans l'année (0 à 366).                                                                                                                                | INT              |



|                                                                       |                                                                                                                                    |                          |
|-----------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|--------------------------|
| <code>EXTRACT (type FROM date)</code>                                 | Extrait une partie d'une date selon un type d'intervalle (comme pour <code>DATE_ADD</code> ).                                      | INT                      |
| <code>FROM_DAYS (n)</code>                                            | Retourne une date à partir d'un nombre de jours (le calendrier année 0 débute à $n=365$ ).                                         | DATE                     |
| <code>FROM_UNIXTIME (nunix[, format])</code>                          | Retourne une date (heure) à partir d'une estampille Unix (nombre de jours depuis le 1/1/1970). Utilisation possible d'un format.   | INT ou DATETIME          |
| <code>HOUR (time)</code>                                              | Extrait l'heure d'un temps.                                                                                                        | INT                      |
| <code>LAST_DAY (date)</code>                                          | Dernier jour du mois d'une date (heure).                                                                                           | DATE                     |
| <code>LOCALTIME, LOCALTIME(), LOCALTIMESTAMP, LOCALTIMESTAMP()</code> | Synonymes de <code>NOW()</code> .                                                                                                  |                          |
| <code>MAKEDATE (annee, njour)</code>                                  | Construit une date à partir d'une année et d'un nombre de jours ( $>0$ , si $njour > 365$ , l'année s'incrémente automatiquement). | DATE                     |
| <code>MAKETIME (heure, minute, seconde)</code>                        | Construit une heure.                                                                                                               | TIME                     |
| <code>MICROSECOND (date)</code>                                       | Extrait les microsecondes d'une date-heure.                                                                                        | INT                      |
| <code>MINUTE (time)</code>                                            | Extrait les minutes d'un temps.                                                                                                    | INT                      |
| <code>MONTH (date), MONTHNAME (date)</code>                           | Retourne respectivement le numéro et le nom du mois d'une date-heure.                                                              | INT, VARCHAR             |
| <code>NOW()</code>                                                    | Date et heure courantes au format 'YYYY-MM-DD HH:MM:SS' ou 'YYYYMMDDHHMMSS'.                                                       | DATETIME ou INT          |
| <code>PERIOD_DIFF (int1, int2)</code>                                 | Nombre de mois séparant les deux dates au format YYMM or YYYYMM.                                                                   | INT                      |
| <code>SECOND (time)</code>                                            | Extrait les secondes d'un temps.                                                                                                   | INT                      |
| <code>SEC_TO_TIME (secondes)</code>                                   | Construit une heure au format 'HH:MM:SS' ou 'HHMMSS'.                                                                              | TIME ou INT              |
| <code>STR_TO_DATE (c, format)</code>                                  | Construit une date (heure) selon un certain format. C'est l'inverse de <code>DATE_FORMAT()</code> .                                | DATE ou DATETIME ou TIME |
| <code>SUBDATE (date, n)</code>                                        | Retranche $n$ jours à une date (heure).                                                                                            | DATE ou DATETIME         |
| <code>SUBTIME (date1, date2)</code>                                   | Retranche <code>date2</code> (TIME) à <code>date1</code> (TIME ou DATETIME).                                                       | TIME ou DATETIME         |
| <code>SYSDATE()</code>                                                | Date et heure courantes au format 'YYYY-MM-DD HH:MM:SS' ou 'YYYYMMDDHHMMSS' (différence avec <code>NOW</code> voir chapitre 1).    | DATETIME ou INT          |
| <code>TIME (datetime)</code>                                          | Extrait le temps d'une date-heure.                                                                                                 | TIME                     |

## 5. Regroupements

Cette section traite à la fois des regroupements de lignes (agrégats) et des fonctions de groupe (ou multilignes). Nous étudierons les parties surlignées de l'instruction SELECT suivante :

```
SELECT [ { DISTINCT | DISTINCTROW } | ALL ] listeColonnes
FROM nomTable [ WHERE condition ]
[ clauseOrdonnancement ]
[ LIMIT [rangDépart,] nbLignes ] ;
```

\_ *listeColonnes* : peut inclure des expressions (présentes dans la clause de regroupement) ou des fonctions de groupe.

\_ *clauseRegroupement* : **GROUP BY** (*expression1*[,*expression2*]...) permet de regrouper des lignes selon la valeur des expressions (colonnes, fonction, constante, calcul).

\_ **HAVING** *condition* : pour inclure ou exclure des lignes aux groupes (la condition ne peut faire intervenir que des expressions du **GROUP BY**).

\_ *ClauseOrdonnancement* : déjà étudié (**ORDER BY** dans la section Projection/Ordonnancement).

Interrogeons la table suivante en composant des regroupements et en appliquant des fonctions de groupe :

**Figure** Table Pilote

Pilote

| brevet | nom              | nbHVol | prime | embauche   | typeAvion | compa |
|--------|------------------|--------|-------|------------|-----------|-------|
| PL-1   | Gratien Viel     | 450    | 500   | 1965-02-05 | A320      | AF    |
| PL-2   | Didier Donsez    | 0      |       | 1995-05-13 | A320      | AF    |
| PL-3   | Richard Grin     | 1000   |       | 2001-09-11 | A320      | SING  |
| PL-4   | Placide Fresnais | 2450   | 500   | 2001-09-21 | A330      | SING  |
| PL-5   | Daniel Vielle    | 400    | 600   | 1965-01-16 | A340      | AF    |
| PL-6   | Francoise Tort   |        | 0     | 2000-12-24 | A340      | CAST  |

## 6. Fonctions de groupe

Nous étudions dans cette section les fonctions usuelles. Le tableau suivant présente les principales fonctions. L'option **DISTINCT** évite les duplicatas (pris en compte sinon par défaut). À l'exception de **COUNT**, toutes les fonctions ignorent les valeurs **NULL** (il faudra utiliser **IFNULL** pour contrer cet effet).

Tableau 14 Fonctions de groupe

| Fonction                              | Objectif                                                                          |
|---------------------------------------|-----------------------------------------------------------------------------------|
| AVG ([DISTINCT] <i>expr</i> )         | Moyenne de <i>expr</i> (nombre).                                                  |
| COUNT ({*   [DISTINCT] <i>expr</i> }) | Nombre de lignes (* toutes les lignes, <i>expr</i> pour les colonnes non nulles). |
| GROUP_CONCAT ( <i>expr</i> )          | Composition d'un ensemble de valeurs.                                             |
| MAX ([DISTINCT] <i>expr</i> )         | Maximum de <i>expr</i> (nombre, date, chaîne).                                    |
| MIN ([DISTINCT] <i>expr</i> )         | Minimum de <i>expr</i> (nombre, date, chaîne).                                    |
| STDDEV ( <i>expr</i> )                | Écart type de <i>expr</i> (nombre).                                               |
| SUM ([DISTINCT] <i>expr</i> )         | Somme de <i>expr</i> (nombre).                                                    |
| VARIANCE ( <i>expr</i> )              | Variance de <i>expr</i> (nombre).                                                 |

Utilisées sans GROUP BY, ces fonctions s'appliquent à la totalité ou à une seule partie d'une table comme le montrent les exemples suivants :

Utilisées sans GROUP BY, ces fonctions s'appliquent à la totalité ou à une seule partie d'une table comme le montrent les exemples suivants :

Tableau 15 Exemples de fonctions de groupe

| Fonction | Exemples                                                                                                                                                                                                                                                                                                                                                                           |
|----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| AVG      | <p>Moyenne des heures de vol et des primes des pilotes de la compagnie 'AF'.</p> <pre>SELECT AVG(nbHVol), AVG(prime) FROM Pilote WHERE compa = 'AF';</pre> <pre> +-----+-----+   AVG(nbHVol)   AVG(prime)   +-----+-----+   283.333333   550.0000   +-----+-----+</pre>                                                                                                            |
| COUNT    | <p>Nombre de pilotes, d'heures de vol et de primes (toutes et distinctes) recensées dans la table.</p> <pre>SELECT COUNT(*), COUNT(nbHVol), COUNT(prime), COUNT(DISTINCT prime) FROM Pilote;</pre> <pre> +-----+-----+-----+-----+   COUNT(*)   COUNT(nbHVol)   COUNT(prime)   COUNT(DISTINCT prime)   +-----+-----+-----+-----+   6   5   4   3   +-----+-----+-----+-----+</pre> |

## 7. Étude du GROUP BY et HAVING

Le groupement de lignes dans une requête se programme au niveau surligné de l'instruction SQL suivante :

```
SELECT col1[, col2...], fonction1Groupe(...)[fonction2Groupe(...)]
```

```
FROM nomTable [ WHERE condition ]
```

```
[ORDER BY {col1 | expr1 | position1} [ASC | DESC] [{col1 ... }]];
```

\_ La clause WHERE de la requête permet d'exclure des lignes pour chaque groupement, ou de rejeter des groupements entiers. Elle s'applique donc à la totalité de la table.

\_ La clause GROUP BY liste les colonnes du groupement.

\_ La clause HAVING permet de poser des conditions sur chaque groupement.

\_ La clause ORDER BY permet de trier le résultat (déjà étudiée).



Les colonnes présentes dans le SELECT doivent apparaître dans le GROUP BY. Seules des fonctions ou expressions peuvent exister en plus dans le SELECT.

Les alias de colonnes ne peuvent pas être utilisés dans la clause GROUP BY.

Dans l'exemple suivant, en groupant sur la colonne compa, trois ensembles de lignes (groupements) sont composés. Il est alors possible d'appliquer des fonctions de groupe à chacun de ces ensembles (dont le nombre n'est pas précisé dans la requête ni limité par le système qui parcourt toute la table).

**Figure** Groupement sur la colonne compa

Pilote

| brevet | nom              | nbHVol | prime | embauche   | typeAvion | compa |
|--------|------------------|--------|-------|------------|-----------|-------|
| PL-1   | Gratien Viel     | 450    | 500   | 1965-02-05 | A320      | AF    |
| PL-2   | Didier Donsez    | 0      |       | 1995-05-13 | A320      | AF    |
| PL-5   | Daniel Vielle    | 400    | 600   | 1965-01-16 | A340      | AF    |
| PL-6   | Francoise Tort   |        | 0     | 2000-12-24 | A340      | CAST  |
| PL-3   | Richard Grin     | 1000   |       | 2001-09-11 | A320      | SING  |
| PL-4   | Placide Fresnais | 2450   | 500   | 2001-09-21 | A330      | SING  |

Tableau 16 Exemples de fonctions de groupe avec GROUP BY

| Fonction | Exemples                                                                                                                                                                                                                                                                                                                                                                                                       |            |             |            |    |            |          |      |      |        |      |             |          |
|----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|-------------|------------|----|------------|----------|------|------|--------|------|-------------|----------|
| AVG      | <p>Moyenne des heures de vol et des primes pour chaque compagnie.</p> <pre>SELECT compa, AVG(nbHVol), AVG(prime) FROM Pilote       GROUP BY (compa) ;</pre> <table><tr><th>compa</th><th>AVG(nbHVol)</th><th>AVG(prime)</th></tr><tr><td>AF</td><td>283.333333</td><td>550.0000</td></tr><tr><td>CAST</td><td>NULL</td><td>0.0000</td></tr><tr><td>SING</td><td>1725.000000</td><td>500.0000</td></tr></table> | compa      | AVG(nbHVol) | AVG(prime) | AF | 283.333333 | 550.0000 | CAST | NULL | 0.0000 | SING | 1725.000000 | 500.0000 |
| compa    | AVG(nbHVol)                                                                                                                                                                                                                                                                                                                                                                                                    | AVG(prime) |             |            |    |            |          |      |      |        |      |             |          |
| AF       | 283.333333                                                                                                                                                                                                                                                                                                                                                                                                     | 550.0000   |             |            |    |            |          |      |      |        |      |             |          |
| CAST     | NULL                                                                                                                                                                                                                                                                                                                                                                                                           | 0.0000     |             |            |    |            |          |      |      |        |      |             |          |
| SING     | 1725.000000                                                                                                                                                                                                                                                                                                                                                                                                    | 500.0000   |             |            |    |            |          |      |      |        |      |             |          |



**COUNT** Nombre de pilotes (et ceux qui ont de l'expérience du vol) par compagnie.

```
SELECT compa, COUNT(*), COUNT(nbHVol) FROM Pilote
      GROUP BY (compa) ;
```

| compa | COUNT(*) | COUNT(nbHVol) |
|-------|----------|---------------|
| AF    | 3        | 3             |
| CAST  | 1        | 0             |
| SING  | 2        | 2             |

**MAX** Nombre d'heures de vol le plus élevé, date d'embauche la plus récente pour chaque compagnie.

```
SELECT compa, MAX(nbHVol), MAX(embauche) "Date+"
      FROM Pilote GROUP BY (compa) ;
```

| compa | MAX(nbHVol) | Date+      |
|-------|-------------|------------|
| AF    | 450.00      | 1995-05-13 |
| CAST  | NULL        | 2000-12-24 |
| SING  | 2450.00     | 2001-09-21 |

**Plusieurs colonnes dans le GROUP BY** Nombre de pilotes qualifiés par type d'appareil et par compagnie.

```
SELECT compa,typeAvion, COUNT(brevet) FROM Pilote
      GROUP BY compa,typeAvion ;
```

| compa | typeAvion | COUNT(brevet) |
|-------|-----------|---------------|
| AF    | A320      | 2             |
| AF    | A340      | 1             |
| CAST  | A340      | 1             |
| SING  | A320      | 1             |
| SING  | A330      | 1             |

## Chapitre 7 : Jointure entre les tables

### I. Introduction

Ce chapitre présente le principe de la jointure entre les tables. Dans ce cours, on va s'intéresser seulement à l'équijointure.

### II. Jointure

#### 1. Définition

Les jointures permettent d'extraire des données issues de plusieurs tables. Le processus de normalisation du modèle relationnel est basé sur la décomposition et a pour conséquence d'augmenter le nombre de tables d'un schéma. Ainsi, la majorité des requêtes utilisent des jointures nécessaires pour pouvoir extraire des données de tables distinctes.

Une jointure est un produit cartésien de deux tables. Elle met en relation deux tables sur la base d'une clause de jointure (comparaison de colonnes). Généralement, cette comparaison fait intervenir une clé étrangère d'une table avec une clé primaire d'une autre table (car le modèle relationnel est fondamentalement basé sur les valeurs).

En SQL, l'expression d'une jointure se fait en précisant le nom des colonnes des tables sur lesquelles on fait la jointure, en désignant les colonnes des différentes tables en écrivant le nom de la table, suivie d'un point puis du nom de la colonne. La clause *WHERE* permet de préciser la qualification de la jointure.

#### 2. Types de Jointure

Bien que, dans le vocabulaire courant, on ne parle que de « jointures » en fonction de la nature de l'opérateur utilisé dans la requête, de la clause de jointure et des tables concernées, on distingue :

- Les jointures internes (inner joins) :
  - L'équijointure (equi join) est la plus connue, elle utilise l'opérateur d'égalité dans la clause de jointure. La jointure naturelle est conditionnée en plus par le nom des colonnes. La non équijointure utilise l'opérateur d'inégalité dans la clause de jointure.
  - L'autojointure (self join) est un cas particulier de l'équijointure, qui met en œuvre deux fois la même table (des alias de tables permettront de distinguer les enregistrements entre eux).

- La jointure externe (outer join), la plus compliquée, qui favorise une table (dite « dominante ») par rapport à l'autre (dite « subordonnée »). Les lignes de la table dominante sont retournées même si elles ne satisfont pas aux conditions de jointure.

Le tableau suivant illustre cette classification sous la forme de quelques conditions appliquées :

| Type de jointure | Syntaxe de la condition                                   |
|------------------|-----------------------------------------------------------|
| Équijointure     | ...WHERE comp = compa;                                    |
| Autojointure     | ...WHERE alias1.chefPil = alias2.brevet;                  |
| Jointure externe | ...FROM Compagnie LEFT OUTER JOIN Pilote ON comp = compa; |

Dans ce cours, on va s'intéresser seulement aux deux premiers types de jointures.

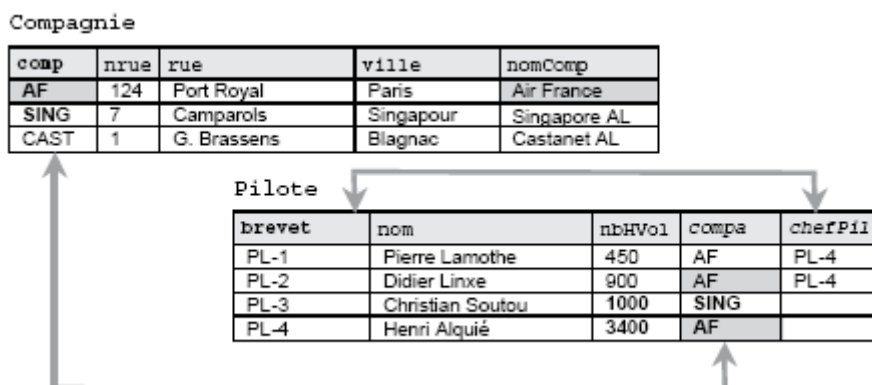
### 1. Équijointure

Une équijointure utilise l'opérateur d'égalité dans la clause de jointure et compare généralement des clés primaires avec des clés étrangères.

En considérant les tables suivantes, les équijointures se programment soit sur les colonnes comp et compa soit sur les colonnes brevet et chefPil. Extrayons par exemple :

- l'identité des pilotes de la compagnie de nom 'Air France' ayant plus de 500 heures de vol (requête R1) ;
- les coordonnées des compagnies qui embauchent des pilotes de moins de 500 heures de vol (requête R2).

La jointure qui résoudra la première requête est illustrée dans la figure par les données grisées, tandis que la deuxième jointure est représentée par les données en gras.



### Écriture « relationnelle »

- MySQL recommande d'utiliser des alias de tables pour améliorer les performances.
- Les alias sont obligatoires quand des colonnes de différentes tables portent le même nom ou dans le cas d'autojointures.

### Écriture « SQL2 »

- La clause JOIN ... ON condition permet de programmer une équijointure.

• L'utilisation de la directive INNER devant JOIN... est optionnelle et est appliquée par défaut.

Le tableau suivant détaille ces requêtes avec les deux syntaxes. Les clauses de jointures sont grisées.

| Requête | Jointure relationnelle                                                                                                                                                   | Jointure SQL2                                                                                                              |
|---------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| R1      | <pre>SELECT brevet, nom FROM Pilote, Compagnie WHERE comp = compa AND nomComp = 'Air France' AND nbHVol &gt; 500;</pre>                                                  | <pre>SELECT brevet, nom FROM Compagnie JOIN Pilote ON comp = compa WHERE nomComp = 'Air France' AND nbHVol &gt; 500;</pre> |
|         | <pre>+-----+-----+   brevet   nom        +-----+-----+   PL-2     Didier Linxe     PL-4     Henri Alquié   +-----+-----+</pre>                                           |                                                                                                                            |
| R2      | <pre>SELECT cpg.nomComp, cpg.nrue,        cpg.rue, cpg.ville FROM Pilote pil, Compagnie cpg WHERE cpg.comp = pil.compa AND pil.nbHVol &lt; 500;</pre>                    | <pre>SELECT nomComp, nrue, rue, ville FROM Compagnie INNER JOIN Pilote ON comp = compa WHERE nbHVol &lt; 500;</pre>        |
|         | <pre>+-----+-----+-----+-----+   nomComp   nrue   rue        ville   +-----+-----+-----+-----+   Air France   124   Port Royal   Paris   +-----+-----+-----+-----+</pre> |                                                                                                                            |

## 2. Autojointure

Cas particulier de l'équijointure, l'autojointure relie une table à elle-même.

Extrayons par exemple :

- L'identité des pilotes placés sous la responsabilité des pilotes de nom 'Alquié' (requête R3) ;
- La somme des heures de vol des pilotes placés sous la responsabilité des chefs pilotes de la compagnie de nom 'Air France' (requête R4).

Ces requêtes doivent être programmées à l'aide d'une autojointure, car elles imposent de parcourir deux fois la table Pilote (examen de chaque pilote en le comparant à un autre).

Les autojointures sont réalisées entre les colonnes brevet et chefPil.

La jointure de la première requête est illustrée dans la figure par les données surlignées en clair, tandis que la deuxième jointure est mise en valeur par les données surlignées en foncé.



Compagnie

| comp | nrue | rue         | ville     | nomComp      |
|------|------|-------------|-----------|--------------|
| AF   | 124  | Port Royal  | Paris     | Air France   |
| SING | 7    | Camparols   | Singapour | Singapore AL |
| CAST | 1    | G. Brassens | Blagnac   | Castanet AL  |

Pilote

| brevet | nom              | nbHVol | compa | chefPil |
|--------|------------------|--------|-------|---------|
| PL-1   | Pierre Lamothe   | 450    | AF    | PL-4    |
| PL-2   | Didier Linxe     | 900    | AF    | PL-4    |
| PL-3   | Christian Soutou | 1000   | SING  |         |
| PL-4   | Henri Alquié     | 3400   | AF    |         |

Le tableau suivant détaille ces requêtes, les clauses d'autojointures sont surlignées. Dans les deux syntaxes, il est impératif d'utiliser un alias de table. Concernant l'écriture « SQL2 », on remarque que les clauses JOIN s'imbriquent (pour joindre plus de deux tables).

| Requête | Jointure relationnelle                                                                                                                                              | Jointure SQL2                                                                                                                                            |
|---------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| R3      | <pre>SELECT p1.brevet, p1.nom FROM Pilote p1, Pilote p2 WHERE p1.chefPil = p2.brevet AND p2.nom LIKE '%Alquié%';</pre>                                              | <pre>SELECT p1.brevet, p1.nom FROM Pilote p1 JOIN Pilote p2 ON p1.chefPil = p2.brevet WHERE p2.nom LIKE '%Alquié%';</pre>                                |
|         | <pre>+-----+-----+   brevet   nom            +-----+-----+   PL-1     Pierre Lamothe     PL-2     Didier Linxe    +-----+-----+</pre>                               |                                                                                                                                                          |
| R4      | <pre>SELECT SUM(p1.nbHVol) FROM Pilote p1, Pilote p2,      Compagnie cpg WHERE p1.chefPil = p2.brevet AND cpg.comp = p2.compa AND cpg.nomComp = 'Air France';</pre> | <pre>SELECT SUM(p1.nbHVol) FROM Pilote p1 JOIN Pilote p2 ON p1.chefPil = p2.brevet JOIN Compagnie ON comp = p2.compa WHERE nomComp = 'Air France';</pre> |
|         | <pre>+-----+   SUM(p1.nbHVol)   +-----+            1350.00   +-----+</pre>                                                                                          |                                                                                                                                                          |

### Inéquijointure

Les requêtes d'inéquijointures font intervenir tout type d'opérateur (<, >, <=, >=, <=>, BETWEEN, LIKE, IN). À l'inverse des équijointures, la clause d'une inéquijointure n'est pas basée sur l'égalité de clés primaires (ou candidates) et de clés étrangères.

En considérant les tables suivantes, extrayons par exemple :

- Les pilotes ayant plus d'expérience que le pilote de numéro de brevet 'PL-2' (requête R5).
- Le titre de qualification des pilotes en raisonnant sur la comparaison des heures de vol avec un ensemble de référence, ici la table HeuresVol (requête R6). Dans notre exemple, il s'agit par exemple de retrouver le fait que le premier pilote est débutant.

La jointure qui résoudra la deuxième requête est illustrée par les niveaux de gris.



Le tableau suivant détaille ces requêtes, les clauses d'inéquijointures sont surlignées :

| Requête | Jointure relationnelle                                                                                                                                                                                                                                  | Jointure SQL2                                                                                                                                           |
|---------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| R5      | SELECT p1.brevet, p1.nom,<br>p1.nbHVol, p2.nbHVol "Référence"<br>FROM Pilote p1, Pilote p2<br>WHERE p1.nbHVol > p2.nbHVol<br>AND p2.brevet = 'PL-2';                                                                                                    | SELECT p1.brevet, p1.nom,<br>p1.nbHVol, p2.nbHVol "Référence"<br>FROM Pilote p1 JOIN<br>Pilote p2 ON p1.nbHVol > p2.nbHVol<br>WHERE p2.brevet = 'PL-2'; |
|         | <pre> +-----+-----+-----+-----+   brevet   nom            nbHVol   Référence   +-----+-----+-----+-----+   PL-3     Christian Soutou   1000.00   900.00       PL-4     Henri Alquié     3400.00   900.00     +-----+-----+-----+-----+           </pre> |                                                                                                                                                         |

| Requête | Jointure relationnelle                                                                                                                                                                                                                                                                                                                                      | Jointure SQL2                                                                                                     |
|---------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| R6      | SELECT pil.brevet, pil.nom,<br>pil.nbHVol, hv.titre<br>FROM Pilote pil, HeuresVol hv<br>WHERE pil.nbHVol BETWEEN<br>hv.basnbHVol AND hv.hautnbHVol;                                                                                                                                                                                                         | SELECT brevet, nom, nbHVol, titre<br>FROM Pilote<br>JOIN HeuresVol ON nbHVol<br>BETWEEN basnbHVol AND hautnbHVol; |
|         | <pre> +-----+-----+-----+-----+   brevet   nom            nbHVol   titre       +-----+-----+-----+-----+   PL-1     Pierre Lamothe   450.00   Débutant      PL-2     Didier Linxe     900.00   Initié        PL-3     Christian Soutou   1000.00   Initié        PL-4     Henri Alquié     3400.00   Expert      +-----+-----+-----+-----+           </pre> |                                                                                                                   |

### III. Les Opérations Ensemblistes

Les opérations ensemblistes en SQL, sont réalisées grâce aux opérateurs:

- UNION
- INTERSECT
- EXCEPT

Ces opérateurs s'utilisent entre deux clauses *SELECT*.

#### 1. L'opérateur UNION

Cet opérateur permet d'effectuer une UNION des tuples sélectionnés par deux clauses *SELECT*.

```
SELECT ---- FROM ---- WHERE -----
```

```
UNION
```

```
SELECT ---- FROM ---- WHERE -----
```

Par défaut les doublons sont automatiquement éliminés. Pour conserver les doublons, il est possible d'utiliser une clause *UNION ALL*.

#### 2. L'opérateur INTERSECT

Cet opérateur permet d'effectuer une INTERSECTION des tuples sélectionnés par deux clauses *SELECT*.

```
SELECT ---- FROM ---- WHERE -----
```

```
INTERSECT
```

```
SELECT ---- FROM ---- WHERE -----
```

L'opérateur *INTERSECT* n'étant pas implémenté dans tous les SGBD, il est possible de le remplacer par des commandes usuelles :

```
SELECT a,b FROM table1
```

```
WHERE EXISTS (SELECT c, d FROM table2
```

```
WHERE a=c AND b=d );
```

### 3. L'opérateur EXCEPT

Cet opérateur permet d'effectuer une DIFFERENCE entre les tuples sélectionnés par deux clauses *SELECT*, c'est-à-dire sélectionner les tuples de la première table n'appartenant pas à la seconde.

```
SELECT a,b FROM table1 WHERE -----
```

```
EXCEPT
```

```
SELECT c,d FROM table2 WHERE -----
```

L'opérateur *EXCEPT* n'étant pas implémenté dans tous les SGBD, il est possible de le remplacer par des commandes usuelles:

```
SELECT a,b FROM table1
```

```
WHERE NOT EXISTS (SELECT c,d FROM table2
```

```
WHERE a=c AND b=d );
```